

Documentación Completa - App Pedidos

Sistema de Gestión de Entregas para Choferes

Índice

1. Información General
2. Especificaciones Técnicas
3. Dependencias y Librerías
4. Estructura del Proyecto
5. Módulos Principales
 - Autenticación y Sesión
 - Kilometraje Obligatorio
 - Inspección Vehicular
 - Gestión de Pedidos
 - Agrupación de Rutas
 - Mapa de Ruta y Exportación
 - Módulo de Vehículos
 - Sistema de Actualización
 - Restricciones de la App
6. Módulos Secundarios
7. Configuración del Servidor
8. Permisos Requeridos
9. API Endpoints
10. Flujo de Trabajo del Usuario

Información General

Nombre del Proyecto: App Pedidos **Tipo de Aplicación:** Aplicación Nativa Android **Lenguaje Principal:** Java 17 **Package Name:** com.example.app_pedidos **Propósito:** Sistema integral de gestión de entregas y logística para choferes con control de kilometraje, inspección vehicular diaria y rutas optimizadas.

Especificaciones Técnicas

Versiones de Android

- **Compilación SDK:** 34 (Android 14)
- **SDK Mínimo:** 24 (Android 7.0 - Nougat)
- **SDK Target:** 34 (Android 14)

Herramientas de Desarrollo

- **Build System:** Gradle 8.7 con Kotlin DSL
- **Android Gradle Plugin:** 8.5.2

- **IDE Recomendado:** Android Studio Hedgehog o superior
- **Java Version:** 17

Arquitectura

- **Patrón de Navegación:** Single Activity Architecture
- **Navigation Component:** AndroidX Navigation con Fragments
- **Binding:** ViewBinding habilitado
- **Arquitectura de UI:** MVVM parcial con ViewModels

Dependencias y Librerías

Networking

Librería	Versión	Uso
Volley	1.2.1	Peticiones HTTP/REST principales
OkHttp	4.12.0	Cliente HTTP avanzado, interceptores

UI/UX

Librería	Versión	Uso
Material Components	Latest	Material Design 3, componentes UI
AndroidX Navigation	2.8.5	Navegación entre fragments
ViewBinding	-	Enlace seguro de vistas
CardView	1.0.0	Cards de pedidos
SwipeRefreshLayout	1.1.0	Pull-to-refresh en listas

Gráficos y Visualización

Librería	Versión	Uso
MPAndroidChart	v3.1.0	Gráficos estadísticos del chofer
Picasso	2.71828	Carga y caché de imágenes

Mapas y Ubicación

Librería	Versión	Uso
Google Play Services Location	21.0.1	Obtención de coordenadas GPS
Mapbox GL JS	v2.15.0	Mapas interactivos (web embebido)

Otras

- AndroidX Core KTX 1.15.0
- AndroidX AppCompat 1.7.0
- ConstraintLayout 2.2.0
- Lifecycle LiveData & ViewModel

Estructura del Proyecto

```
\\192.168.60.117\AndroidStudioProjects\New_App_Pedidos\
|
|├── app/
|│   ├── src/main/
|│   │   ├── java/com/example/app_pedidos/
|│   │   │   ├── MainActivity.java                # Activity principal con
|│   │   │   ├── DrawerLayout
|│   │   │   │   ├── ApiConfig.java                # Configuración URL base del
|│   │   │   │   │   ├── servidor
|│   │   │   │   │   │   ├── ConexionPHP.java        # Utilidades de conexión HTTP
|│   │   │   │   │   │   ├── UpdateManager.java       # Gestor de actualizaciones
|│   │   │   │   │   │   ├── OTA
|│   │   │   │   │   │   │   ├── network/
|│   │   │   │   │   │   │   │   ├── Utf8JsonObjectRequest.java    # Clases de peticiones HTTP
|│   │   │   │   │   │   │   │   │   ├── Utf8JsonArrayRequest.java # Request JSON con encoding
|│   │   │   │   │   │   │   │   │   ├── Utf8StringRequest.java
|│   │   │   │   │   │   │   │   │   ├── UTF-8
|│   │   │   │   │   │   │   │   │   │   ├── Utf8JsonArrayRequest.java
|│   │   │   │   │   │   │   │   │   │   ├── Utf8StringRequest.java
|│   │   │   │   │   │   │   │   │   ├── ui/
|│   │   │   │   │   │   │   │   │   │   ├── Login/
|│   │   │   │   │   │   │   │   │   │   │   ├── LoginActivity.java    # Pantalla de autenticación
|│   │   │   │   │   │   │   │   │   │   │   ├── LoginViewModel.java
|│   │   │   │   │   │   │   │   │   │   ├── home/
|│   │   │   │   │   │   │   │   │   │   │   ├── HomeFragment.java      # Lista de pedidos activos
|│   │   │   │   │   │   │   │   │   │   │   ├── HomeViewModel.java
|│   │   │   │   │   │   │   │   │   │   │   ├── PedidoAdapter.java     # Adapter para RecyclerView
|│   │   │   │   │   │   │   │   │   │   ├── Pedido/
|│   │   │   │   │   │   │   │   │   │   │   ├── DetallePedidoActivity.java # Gestión de pedidos
|│   │   │   │   │   │   │   │   │   │   │   │   ├── individuales
|│   │   │   │   │   │   │   │   │   │   │   │   │   ├── DetallePedidoActivity.java # Detalles y acciones del
|│   │   │   │   │   │   │   │   │   │   │   │   │   │   ├── pedido
|│   │   │   │   │   │   │   │   │   │   │   │   │   │   ├── Detalle_Actualizaciones.java # Historial de cambios
|│   │   │   │   │   │   │   │   │   │   │   │   │   │   ├── GrupoRutaActivity.java    # Vista de grupo de pedidos
|│   │   │   │   │   │   │   │   │   │   │   │   │   │   ├── GrupoMapaDialogFragment.java # Mapa del grupo con Mapbox
|│   │   │   │   │   │   │   │   │   │   │   │   │   │   ├── HtmlBuilder.java        # Constructor de HTML para
|│   │   │   │   │   │   │   │   │   │   │   │   │   │   ├── mapas
|│   │   │   │   │   │   │   │   │   │   │   │   │   │   ├── Inspeccion/
|│   │   │   │   │   │   │   │   │   │   │   │   │   │   │   ├── InspeccionVehicularActivity.java # Inspección vehicular diaria
|│   │   │   │   │   │   │   │   │   │   │   │   │   │   │   ├── InspeccionHoyActivity.java # Vista de solo lectura
|│   │   │   │   │   │   │   │   │   │   │   │   │   │   │   └── InspeccionAdapter.java
```

	vehicle/	# Información del vehículo
	├─ VehicleFragment.java	
	└─ VehicleViewModel.java	
	Estadisticas/	# Métricas de desempeño
	├─ EstadisticasFragment.java	
	└─ EstadisticasViewModel.java	
	slideshow/	# Historial de pedidos
	├─ SlideshowFragment.java	
	└─ SlideshowViewModel.java	
	common/	# Componentes compartidos
	util/	# Utilidades generales
	├─ NotificationHelper.java	# Gestión de notificaciones
	└─ Events.java	# Constantes de broadcast
events		
	res/	
	└─ layout/	# 30+ archivos XML de
interfaz		
	├─ activity_main.xml	
	├─ activity_login.xml	
	├─ activity_detalle_pedido.xml	
	├─ activity_inspeccion_vehicular.xml	
	├─ dialog_kilometraje.xml	# Modal de captura de km
	├─ dialog_grupo_mapa.xml	
	└─ ...	
	drawable/	# Iconos y recursos gráficos
	values/	
	├─ strings.xml	# Textos y tokens API
	├─ colors.xml	
	└─ themes.xml	
	menu/	# Menús de navegación
	├─ activity_main_drawer.xml	# Drawer navigation menu
	└─ main.xml	
	navigation/	# Grafos de navegación
	└─ mobile_navigation.xml	
	AndroidManifest.xml	# Configuración de la app
	build.gradle.kts	# Configuración de build
	gradle/	
	└─ libs.versions.toml	# Catálogo de versiones
	build.gradle.kts	# Configuración raíz
	settings.gradle.kts	# Configuración del proyecto

Módulos Principales

1. Módulo de Autenticación y Sesión

Ubicación

ui/Login/LoginActivity.java

Funcionalidades

1.1 Login con Credenciales

- Formulario con campos de usuario y contraseña
- Validación de campos vacíos
- Autenticación contra API REST del servidor
- Mensajes de error descriptivos

1.2 Persistencia de Sesión

- Almacenamiento en **SharedPreferences**:
 - `username` - Nombre de usuario
 - `userId` - ID del usuario en BD
 - `sucursalId` - Sucursal asignada
 - `vehiculo_id` - ID del vehículo asignado
 - `isLoggedIn` - Estado de sesión (booleano)

1.3 Auto-Login

- Verificación automática al abrir la app
- Si existe sesión activa válida, dirige directamente a MainActivity
- No solicita credenciales nuevamente

1.4 Detección de Cambio de Vehículo

- Banner de notificación si el vehículo asignado cambió desde el último login
- Comparación del `vehiculo_id` almacenado vs. el devuelto por el servidor

1.5 Redirección Post-Login

- Transición automática a `MainActivity` tras autenticación exitosa
- Pasa `username` como Extra en el Intent

API Endpoint

```
POST /Pedidos_GA/App/login.php
```

Request Body (JSON):

```
{
  "username": "string",
  "password": "string"
}

Response (JSON):
{
  "success": true/false,
  "message": "string",
  "userId": int,
  "sucursalId": int,
  "vehiculo_id": int
}
```

Layout

`res/layout/activity_login.xml`

2. Módulo de Kilometraje Obligatorio

Ubicación

`MainActivity.java` (líneas 176-310)

Funcionalidades

2.1 Modal de Captura Obligatoria

- **Dialog modal no cancelable** que bloquea completamente el acceso a la aplicación
- Se muestra automáticamente al iniciar la app
- No permite cerrar con botón "Atrás" o tocando fuera del dialog
- Campos:
 - **Fecha y hora actual** (auto-llenado, no editable)
 - **Kilometraje inicial del día** (input numérico)
 - Botón "Registrar"

2.2 Validación Horaria

- **Solo obligatorio después de las 8:00 AM**
- **Excepción:** No se solicita los domingos
- Lógica:

```
Calendar ahora = Calendar.getInstance();
int hora = ahora.get(Calendar.HOUR_OF_DAY);
int diaSemana = ahora.get(Calendar.DAY_OF_WEEK);

if (hora >= 8 && diaSemana != Calendar.SUNDAY) {
```

```
// Solicitar kilometraje  
}
```

2.3 Validación de Incremento

- El kilometraje ingresado **NO puede ser menor** al último registrado
- Obtiene `lastKmFinal` desde `SharedPreferences`
- Muestra Toast de error si no cumple la validación
- Ejemplo:

```
Último km registrado: 15,500  
Nuevo km ingresado: 15,300 ✗ Error  
Nuevo km ingresado: 15,600 ✓ Válido
```

2.4 Verificación de Registro Diario

- Consulta al servidor: ¿Ya se registró kilometraje hoy?
- Estados posibles:
 - `debe_capturar = true` → Muestra modal
 - `debe_capturar = false` → Continúa normalmente
 - Error de conexión → Opción de reintentar o cerrar sesión

2.5 Persistencia y Reintentos

- Almacena `lastKmFinal` tras registro exitoso
- Usa `lastKmFinal` como validación para próximos registros
- En caso de error de red:
 - Muestra dialog de error
 - Botón "Reintentar" → Vuelve a intentar el registro
 - Botón "Cerrar Sesión" → Logout y regreso a `LoginActivity`

2.6 Integración con Inspección

- Tras registrar kilometraje exitosamente, lanza automáticamente la verificación de inspección vehicular
- Flujo: Kilometraje → Inspección → Desbloqueo de app

🔑 Métodos Clave

Método	Descripción
<code>verificarEstadoKilometraje()</code>	Consulta al servidor si debe capturar km hoy
<code>mostrarDialogoCapturaKilometraje()</code>	Muestra el dialog modal de captura
<code>registrarKilometrajeForzado(kmInicial)</code>	Envía el kilometraje al servidor

🔗 API Endpoints

Estado de Kilometraje

```
GET /Pedidos_GA/App/estado_kilometraje.php?username={username}
```

Response (JSON):

```
{
  "debe_capturar": boolean,
  "ultimo_km": int,
  "fecha_ultimo_registro": "YYYY-MM-DD"
}
```

Registro de Kilometraje

```
POST /Pedidos_GA/App/registrar_kilometraje.php
```

Request Body (JSON):

```
{
  "username": "string",
  "km_inicial": int,
  "fecha": "YYYY-MM-DD HH:mm:ss"
}
```

Response (JSON):

```
{
  "success": true/false,
  "message": "string"
}
```

Layout

[res/layout/dialog_kilometraje.xml](#)

Experiencia de Usuario

1. Usuario abre la app a las 8:30 AM (lunes)
2. MainActivity detecta hora $\geq 8:00$ y día $\neq$ domingo
3. Consulta servidor: ¿Ya registró km hoy?
4. Servidor responde: `debe_capturar = true`
5. Se muestra dialog modal bloqueante
6. Usuario ingresa 15,600 km
7. Sistema valida: $15,600 \geq 15,500$ (último registrado) ☒
8. Envía a servidor
9. Servidor responde: `success = true`
10. Dialog se cierra automáticamente
11. Guarda `lastKmFinal = 15600` en SharedPreferences
12. Lanza verificación de inspección vehicular

3. Formulario de Inspección Vehicular

Ubicación

- **Activity Principal:** `ui/Inspeccion/InspeccionVehicularActivity.java`
- **Vista de Solo Lectura:** `ui/Inspeccion/InspeccionHoyActivity.java`

Funcionalidades

3.1 Checklist Completo por Secciones

La inspección está organizada en **7 secciones principales** con un total de **29 ítems** a verificar:

#	Sección	Cantidad de Ítems	Items Críticos
1	Sistema de Luces	6	Luces frontales, traseras, direccionales
2	Parte Externa	8	Carrocería, espejos, parabrisas
3	Parte Interna	8	Cinturones, tablero, aire acondicionado
4	Estado de Llantas	2	Desgaste, presión
5	Accesorios de Seguridad	2	Extintor, botiquín
6	Tapas y Otros	2	Tapa de combustible, capó
7	Rotulado	1	Logos y rotulación del vehículo

Total: 29 puntos de inspección

3.2 Sistema de Calificación

Cada ítem puede tener 3 estados:

Estado	Color	Significado
Bien	Verde	El componente está en condiciones óptimas
Mal	Rojo	El componente tiene problemas o está defectuoso
N/A	Gris	No aplica para este vehículo

3.3 Observaciones Obligatorias

- Cuando un ítem se marca como **"Mal"**, se despliega automáticamente un campo de texto
- Este campo es **obligatorio** y debe describir el problema específico
- Validación: No permite enviar el formulario si hay ítems "Mal" sin observaciones
- Ejemplo de observación: "Luz trasera izquierda fundida, requiere reemplazo"

3.4 Asignación Masiva por Sección

Cada sección tiene **3 botones radio** en el encabezado:

- ☒ **Bien** - Marca todos los ítems de la sección como "Bien"
- ☒ **Mal** - Marca todos los ítems de la sección como "Mal"
- ☐ **N/A** - Marca todos los ítems de la sección como "N/A"

Ventaja: Permite inspecciones más rápidas cuando una sección completa está en buen estado

3.5 Ítems Críticos con Marcador (NN)**

Los ítems marcados con **(NN)*** son críticos y requieren especial atención:

- Luces frontales (NN)**
- Luces traseras (NN)**
- Cinturones de seguridad (NN)**
- Frenos (NN)**
- Direccionales (NN)**

Estos ítems son esenciales para la seguridad vial.

3.6 Visual Feedback

- **Ítems auto-asignados** (mediante botones de sección): Fondo azul claro (#E3F2FD)
- **Ítems asignados manualmente:** Fondo blanco
- **Botón "Enviar"** habilitado solo cuando todos los ítems tienen calificación

3.7 Validación Completa

Antes de enviar, el sistema verifica:

1. ☒ Todos los 29 ítems tienen calificación (Bien/Mal/N/A)
2. ☒ Todos los ítems marcados como "Mal" tienen observaciones
3. ☒ Las observaciones tienen mínimo 3 caracteres
4. ☒ Si falta algo, muestra Toast indicando qué falta

3.8 Envío al Servidor

Una vez validado:

- Construye JSON con toda la inspección
- Incluye: fecha, hora, username, vehiculo_id, sección, ítem, estado, observaciones
- Envía petición POST
- Muestra Toast de confirmación
- Cierra la Activity y regresa a MainActivity

3.9 Vista de Solo Lectura (InspeccionHoyActivity)

- Consulta la inspección del día actual
- Muestra todos los ítems con sus calificaciones
- No permite editar

- Accesible desde el módulo de Vehículos
- Útil para consultas posteriores en el día

Detalle de las Secciones

Sección 1: Sistema de Luces

1. Luces frontales (NN)**
2. Luces traseras (NN)**
3. Luces de freno
4. Direccionales (NN)**
5. Luces de reversa
6. Luces internas

Sección 2: Parte Externa

1. Estado de carrocería
2. Espejos laterales
3. Parabrisas delantero
4. Parabrisas trasero
5. Limpiaparabrisas
6. Antena
7. Placa delantera
8. Placa trasera

Sección 3: Parte Interna

1. Cinturones de seguridad (NN)**
2. Tablero de instrumentos
3. Aire acondicionado
4. Calefacción
5. Radio/audio
6. Claxon
7. Frenos (NN)**
8. Volante

Sección 4: Estado de Llantas

1. Desgaste de llantas
2. Presión de llantas

Sección 5: Accesorios de Seguridad

1. Extintor
2. Botiquín de primeros auxilios

Sección 6: Tapas y Otros

1. Tapa de combustible
2. Capó

Sección 7: Rotulado

1. Logos y rotulación del vehículo

🔗 API Endpoints

Guardar Inspección

POST /Pedidos_GA/App/guardar_checklist.php

Request Body (JSON):

```
{
  "username": "string",
  "vehiculo_id": int,
  "fecha": "YYYY-MM-DD",
  "hora": "HH:mm:ss",
  "items": [
    {
      "seccion": "Sistema de Luces",
      "item": "Luces frontales (NN)**",
      "estado": "Bien|Mal|N/A",
      "observaciones": "string o null"
    },
    ...
  ]
}
```

Response (JSON):

```
{
  "success": true/false,
  "message": "string"
}
```

Obtener Inspección del Día

GET /Pedidos_GA/App/obtener_checklist_hoy.php?username={username}&fecha={YYYY-MM-DD}

Response (JSON):

```
{
  "success": true/false,
  "items": [
    {
      "seccion": "string",
      "item": "string",
      "estado": "string",

```

```

        "observaciones": "string"
    },
    ...
]
}

```

Layouts

Archivo	Descripción
<code>activity_inspeccion_vehicular.xml</code>	Layout principal del formulario
<code>layout_inspeccion_header.xml</code>	Encabezado de sección con botones radio
<code>layout_inspeccion_section.xml</code>	Container de cada sección
<code>layout_inspeccion_item.xml</code>	Item individual con RadioGroup
<code>layout_inspeccion_observaciones.xml</code>	Campo de observaciones desplegable

Integración con MainActivity

- Lanzada automáticamente tras registro de kilometraje exitoso
- Verifica si ya se realizó la inspección hoy
- Si no existe inspección del día:
 - Bloquea acceso al DrawerLayout
 - Lanza `InspeccionVehicularActivity`
- Si existe inspección:
 - Desbloquea la app
 - Permite uso normal

Condiciones de lanzamiento:

- Hora >= 8:00 AM
- Día != Domingo
- No hay inspección registrada hoy

Experiencia de Usuario

1. Usuario completa registro de kilometraje
2. MainActivity verifica: ¿Existe inspección hoy?
3. Servidor responde: No
4. Se lanza `InspeccionVehicularActivity`
5. Usuario ve formulario con 7 secciones
6. Usuario revisa "Sistema de Luces" → Todo bien
7. Presiona botón  "Bien" en el header de la sección
8. Todos los 6 ítems de luces se marcan automáticamente como "Bien"
9. Fondo de los ítems cambia a azul claro
10. Usuario revisa "Parte Externa"
11. Encuentra espejo lateral derecho roto

12. Marca "Bien" para 7 ítems
 13. Marca "Mal" para "Espejos laterales"
 14. Aparece campo de observaciones
 15. Escribe: "Espejo lateral derecho tiene grieta en el cristal"
 16. Continúa con las demás secciones
 17. Al finalizar, presiona "Enviar"
 18. Sistema valida: ☒ Todos los ítems calificados, ☒ Observación completa
 19. Envía JSON al servidor
 20. Servidor responde: `success = true`
 21. Muestra Toast: "Inspección registrada exitosamente"
 22. Cierra Activity
 23. MainActivity desbloquea el drawer
 24. Usuario puede usar la app normalmente
-

4. Gestión de Pedidos

Ubicación

`ui/Pedido/DetallePedidoActivity.java`

Funcionalidades

4.1 Visualización de Detalles Completos

La pantalla de detalle del pedido muestra toda la información relevante:

Información General:

- **ID del Pedido** (número único)
- **Sucursal** origen
- **Cliente** (nombre completo o razón social)
- **Estado actual** (badge con color)
- **Grupo de ruta** (si aplica)

Fechas:

- **Fecha de recepción** del pedido
- **Fecha de entrega** programada
- **Hora de entrega** estimada

Personas Involucradas:

- **Chofer** asignado
- **Vendedor** responsable
- **Contacto** en destino

Información de Entrega:

- **Dirección completa** de entrega

- **Teléfono** de contacto
- **Ventanas horarias** (Ej: "9:00 AM - 12:00 PM")

Información Comercial:

- **Número de factura**
- **Monto** (si aplica)

Observaciones:

- **Comentarios** del vendedor o despacho
- **Instrucciones especiales** de entrega

4.2 Actualización de Estado

Estados disponibles:

Estado	Color	Descripción
ACTIVO	Azul (#1976D2)	Pedido recibido, pendiente de salir a ruta
EN RUTA	Naranja (#EF6C00)	Chofer en camino al destino
REPROGRAMADO	Morado (#8E24AA)	Entrega reagendada para otra fecha
EN TIENDA	Amarillo (#FBC02D)	Cliente pasará a recoger a sucursal
ENTREGADO	Verde (#2E7D32)	Entrega completada exitosamente
CANCELADO	Rojo (#C62828)	Pedido cancelado

Proceso de actualización:

1. Usuario presiona botón "Actualizar Estado"
2. Se muestra **Dialog con RadioGroup** de estados
3. Usuario selecciona el nuevo estado
4. Sistema captura automáticamente:
 - **Coordenadas GPS** actuales (latitud, longitud)
 - **Fecha y hora** exactas del cambio
 - **Username** del chofer
5. Envía petición al servidor
6. Actualiza UI local
7. **Emite broadcast** ACTION_PEDIDO_ESTADO_ACTUALIZADO
8. Todos los fragments suscritos se refrescan automáticamente

Validaciones:

- No permite cambiar de "ENTREGADO" a otro estado (regla de negocio)
- Requiere permisos de ubicación activos
- Si falla la captura de GPS, usa última ubicación conocida

4.3 Captura de Evidencia Fotográfica

Funcionalidad:

- Botón "Subir Foto" con ícono de cámara
- Al presionar, abre selector de archivos del dispositivo
- Permite seleccionar imagen desde:
 - Galería de fotos
 - Google Photos
 - Aplicación de cámara (tomar foto nueva)

Proceso de upload:

1. Usuario selecciona imagen (JPG, PNG)
2. Sistema lee el archivo como Bitmap
3. Comprime la imagen (calidad 80%)
4. Convierte a Base64
5. Crea petición Multipart con:
 - `pedido_id`
 - `username`
 - `image` (Base64)
 - `tipo` = "evidencia"
6. Envía al servidor
7. Servidor almacena en `/uploads/evidencias/`
8. Actualiza campo `foto_evidencia` en BD

Visual Feedback:

- Antes de subir: Botón con fondo gris
- Tras verificar existencia: Botón con fondo verde
- Durante upload: ProgressDialog
- Tras éxito: Toast confirmación + botón se pone verde

Verificación al abrir:

- Al cargar el detalle del pedido, consulta al servidor: ¿Existe foto para este pedido?
- Si existe: Botón verde + texto "Foto subida"
- Si no existe: Botón gris + texto "Subir foto"

4.4 Navegación a Google Maps**Funcionalidad:**

- Botón "Abrir en Google Maps" con ícono de mapa
- Genera URL de ruta desde origen a destino

Proceso:

1. Obtiene coordenadas almacenadas en BD:
 - `origen_lat`, `origen_lng` (ubicación de sucursal)
 - `destino_lat`, `destino_lng` (ubicación del cliente)
2. Construye URL:


```
https://www.google.com/maps/dir/?api=1
&origin={origen_lat},{origen_lng}
&destination={destino_lat},{destino_lng}
&travelmode=driving
```

3. Crea Intent con **ACTION_VIEW**
4. Abre app de Google Maps (o navegador si no está instalada)
5. Google Maps muestra ruta optimizada con indicaciones turn-by-turn

Ventaja:

- Navegación GPS en tiempo real
- Tráfico en vivo
- Alertas de accidentes
- Rutas alternativas

4.5 Descarga de Documentos**Funcionalidad:**

- Botón "Descargar Documento" (usualmente la factura PDF)
- Verifica existencia antes de mostrar el botón

Proceso de verificación:

1. Al cargar el detalle, envía petición GET:

```
/Pedidos_GA/App/verificar_documento.php?pedido_id={id}
```

2. Servidor responde:

```
{
  "existe": true/false,
  "url": "http://servidor/documentos/factura_12345.pdf"
}
```

3. Si existe: Botón verde + texto "Descargar PDF"
4. Si no existe: Botón oculto o deshabilitado

Proceso de descarga:

1. Usuario presiona botón
2. Crea **DownloadManager.Request** con:
 - URL del documento
 - Título: "Factura_{pedido_id}"
 - Descripción: "Descargando documento..."

- Destino: Carpeta Downloads pública
- 3. Muestra notificación de descarga en progreso
- 4. Al completar:
 - Notificación cambia a "Descarga completada"
 - Botón para abrir el PDF
- 5. Abre con visor de PDFs predeterminado

4.6 Historial de Actualizaciones

Funcionalidad:

- Botón "Ver Detalles" o "Historial"
- Abre `Detalle_Actualizaciones.java`
- Muestra **timeline** de todos los cambios de estado del pedido

Información mostrada:

Campo	Descripción
Fecha y hora	Timestamp exacto del cambio
Estado anterior	Estado antes del cambio
Estado nuevo	Estado después del cambio
Usuario	Username del chofer que hizo el cambio
Coordenadas	Ubicación GPS donde se realizó el cambio
Observaciones	Comentarios opcionales

Formato visual:

- Lista vertical tipo timeline
- Cada cambio es un card
- Iconos de estado con colores
- Orden: Más reciente primero

Utilidad:

- Auditoría completa del pedido
- Resolución de disputas
- Análisis de tiempos de entrega
- Tracking preciso del pedido

🔗 API Endpoints

Actualizar Estado

```
POST /Pedidos_GA/App/actualizar_estado.php
```

```
Request Body (JSON):
{
  "pedido_id": int,
  "estado": "string",
  "username": "string",
  "latitud": double,
  "longitud": double,
  "fecha": "YYYY-MM-DD HH:mm:ss",
  "observaciones": "string" (opcional)
}

Response (JSON):
{
  "success": true/false,
  "message": "string"
}
```

Subir Foto

POST /Pedidos_GA/App/subir_foto.php

Request Body (Multipart):

- pedido_id: int
- username: string
- image: string (Base64)
- tipo: "evidencia"

Response (JSON):

```
{
  "success": true/false,
  "message": "string",
  "filename": "string"
}
```

Verificar Foto

GET /Pedidos_GA/App/verificar_foto.php?pedido_id={id}

Response (JSON):

```
{
  "existe": boolean,
  "url": "string"
}
```

Descargar Documento

```
GET /Pedidos_GA/App/descargar_documento.php?pedido_id={id}
```

Response:

- Content-Type: application/pdf
- Archivo PDF directo

Historial de Actualizaciones

```
GET /Pedidos_GA/App/historial_pedido.php?pedido_id={id}
```

Response (JSON):

```
{
  "success": true,
  "historial": [
    {
      "fecha": "YYYY-MM-DD HH:mm:ss",
      "estado_anterior": "string",
      "estado_nuevo": "string",
      "usuario": "string",
      "latitud": double,
      "longitud": double,
      "observaciones": "string"
    },
    ...
  ]
}
```

Layouts

Archivo	Descripción
activity_detalle_pedido.xml	Layout principal del detalle
detalle_actualizaciones.xml	Timeline de historial

Experiencia de Usuario

Caso de uso típico:

1. Chofer abre lista de pedidos activos
2. Selecciona pedido #12345
3. Se abre [DetallePedidoActivity](#)
4. Ve todos los detalles: Cliente "Tienda ABC", Dirección "Calle 123"
5. Presiona "Abrir en Google Maps"
6. Google Maps abre con ruta trazada
7. Sigue la ruta hasta llegar al destino
8. Al llegar, regresa a la app

- 9. Presiona "Actualizar Estado"
- 10. Selecciona "EN RUTA" → "ENTREGADO"
- 11. Sistema captura GPS: 19.4326, -99.1332
- 12. Envía actualización al servidor
- 13. Toast: "Estado actualizado correctamente"
- 14. Badge del pedido cambia a verde "ENTREGADO"
- 15. Presiona "Subir Foto"
- 16. Toma foto del comprobante de entrega
- 17. Sistema la sube al servidor
- 18. Toast: "Foto subida exitosamente"
- 19. Botón "Subir Foto" se pone verde
- 20. Presiona "Ver Historial"
- 21. Ve timeline:
 - 08:30 → ACTIVO (Despacho)
 - 09:15 → EN RUTA (GPS: Sucursal)
 - 10:45 → ENTREGADO (GPS: Cliente)
- 22. Regresa a lista de pedidos
- 23. HomeFragment se refresca automáticamente (broadcast)
- 24. Pedido ya no aparece en lista de activos

5. Agrupación y Gestión de Rutas

Ubicación

ui/Pedido/GrupoRutaActivity.java

Funcionalidades

5.1 Visualización de Grupo de Pedidos

Los pedidos pueden agruparse en "rutas" para optimizar las entregas. Esta pantalla muestra todos los pedidos de un grupo específico.

Header del Grupo:

Campo	Descripción
Nombre del Grupo	Ej: "Ruta Norte - Lunes AM"
Sucursal	Sucursal de origen
Chofer	Nombre del chofer asignado
Estado del Grupo	Activo / En progreso / Completado
Fecha de ruta	Fecha programada para el grupo
Cantidad de pedidos	Total de pedidos en el grupo

Lista de Pedidos:

- Cards ordenados según orden de entrega
- Cada card muestra:
 - Número de orden (1, 2, 3...)
 - ID del pedido
 - Cliente
 - Dirección resumida
 - Estado actual (con color)
 - Hora estimada de entrega
- **Pull-to-refresh** para actualizar la lista

5.2 Ordenamiento Inteligente

Algoritmo de ordenamiento:

1. **Prioridad 1:** Campo `orden_entrega` (definido por el despachador)
 - Respeta el orden planificado para optimizar la ruta
 - Ejemplo: Si el despachador definió que el pedido #105 debe entregarse antes que #103, se respeta ese orden
2. **Prioridad 2:** ID de pedido (criterio de desempate)
 - Si dos pedidos tienen el mismo `orden_entrega`, se ordena por ID ascendente
3. **Eliminación automática de duplicados**
 - Si un pedido aparece varias veces en la respuesta del servidor, solo muestra una instancia
 - Usa `LinkedHashMap` con `pedido_id` como key

Código del ordenamiento:

```
Collections.sort(pedidosList, new Comparator<Pedido>() {  
    @Override  
    public int compare(Pedido p1, Pedido p2) {  
        // Primero comparar por orden_entrega  
        int ordenComparison = Integer.compare(p1.getOrdenEntrega(),  
p2.getOrdenEntrega());  
        if (ordenComparison != 0) {  
            return ordenComparison;  
        }  
        // Si son iguales, comparar por ID  
        return Integer.compare(p1.getId(), p2.getId());  
    }  
});
```

5.3 Colores por Estado

Cada card tiene un borde coloreado según el estado del pedido:

Estado	Color	Hex
ACTIVO	Azul	#1976D2
EN RUTA	Naranja	#EF6C00
REPROGRAMADO	Morado	#8E24AA
EN TIENDA	Amarillo	#FBC02D
ENTREGADO	Verde	#2E7D32
CANCELADO	Rojo	#C62828

Ventaja visual: El chofer puede identificar rápidamente el progreso del grupo

5.4 Interacción con Pedidos Individuales

- Al tocar un card de pedido:
 - Abre `DetallePedidoActivity` con toda la información
 - Permite actualizar estado, subir foto, etc.
 - Al regresar al grupo, la lista se refresca automáticamente

5.5 Botón "Ver Mapa del Grupo"

- Botón flotante (FAB) en la esquina inferior derecha
- Ícono:  mapa
- Al presionar:
 - Abre `GrupoMapaDialogFragment`
 - Muestra mapa interactivo con todos los pedidos del grupo
 - Ver detalle completo en [Módulo 6](#)

5.6 Actualización Automática

Métodos de actualización:

- Pull-to-refresh:** Usuario desliza hacia abajo
- BroadcastReceiver:** Escucha `ACTION_PEDIDO_ESTADO_ACTUALIZADO`
 - Si algún pedido del grupo cambia de estado en otra pantalla, esta lista se actualiza sola
- onResume():** Al volver de otra Activity, refresca la lista

Límite de consulta:

- Consulta hasta 1000 pedidos por grupo (límite en query SQL)
- En la práctica, los grupos suelen tener 5-20 pedidos

🔑 API Endpoint

```
GET /Pedidos_GA/App/Consultar.php?grupo_id={id}&limit=1000
```

Response (JSON):

```
{
  "success": true,
  "grupo": {
    "id": int,
    "nombre": "string",
    "sucursal": "string",
    "chofer": "string",
    "estado": "string",
    "fecha": "YYYY-MM-DD"
  },
  "pedidos": [
    {
      "id": int,
      "orden_entrega": int,
      "cliente": "string",
      "direccion": "string",
      "estado": "string",
      "hora_estimada": "HH:mm",
      "latitud": double,
      "longitud": double,
      "factura": "string"
    },
    ...
  ]
}
```

Layout

activity_grupo_ruta.xml

Experiencia de Usuario

1. Chofer ve en HomeFragment que tiene 8 pedidos en "Ruta Norte - Lunes AM"
2. Toca el grupo
3. Se abre **GrupoRutaActivity**
4. Ve header:
 - Ruta Norte - Lunes AM
 - Sucursal: Centro
 - Chofer: Juan Pérez
 - 8 pedidos
5. Ve lista ordenada:
 - **1** Pedido #101 - Tienda ABC - ACTIVO (azul)
 - **2** Pedido #103 - Mercado XYZ - ACTIVO (azul)
 - **3** Pedido #105 - Farmacia 123 - EN RUTA (naranja)
 - ...
6. Toca pedido #101
7. Abre detalle, actualiza a "EN RUTA", sube foto
8. Regresa al grupo
9. Lista se refresca automáticamente

10. Ahora #101 aparece en naranja "EN RUTA"
 11. Presiona FAB "Ver Mapa del Grupo"
 12. Se abre dialog con mapa interactivo (ver siguiente módulo)
-

6. Mapa de Ruta y Exportación a Google Maps

Ubicación

- **Fragment Principal:** `ui/Pedido/GrupoMapaDialogFragment.java`
- **Builder de HTML:** `ui/Pedido/HtmlBuilder.java`

Funcionalidades

6.1 Mapa Interactivo con Mapbox

Se genera dinámicamente un **mapa HTML embebido** usando **Mapbox GL JS v2.15.0**.

Características técnicas:

- Motor de renderizado: WebGL (aceleración por GPU)
- Estilo de mapa: `mapbox://styles/mapbox/streets-v11`
- Idioma: Español
- Zoom inicial: Ajuste automático para mostrar todos los pedidos
- Control de zoom: Botones + / -
- Control de brújula: Rotación del mapa

Componentes visualizados:

1. Markers numerados:

- Cada pedido tiene un marcador circular
- Número interno indica orden de entrega (1, 2, 3...)
- Color del marcador según estado del pedido
- Tamaño: 30px de diámetro
- Texto blanco en negrita

2. Colores por estado:

Estado	Color	Hex
ACTIVO	Azul	#1976D2
EN RUTA	Naranja	#EF6C00
REPROGRAMADO	Morado	#8E24AA
EN TIENDA	Amarillo	#FBC02D
ENTREGADO	Verde	#2E7D32
CANCELADO	Rojo	#C62828

3. Popups informativos:

- Al hacer clic en un marcador, se abre un popup
- Contenido:
 - **Factura:** #12345
 - **Cliente:** Tienda ABC
 - **Dirección:** Calle 123, Colonia Centro
 - **Estado:** EN RUTA
- Diseño: Card blanco con sombra, texto legible

4. Trazo de ruta general:

- Línea que conecta todos los pedidos en orden
- Color: Gris claro (#9E9E9E)
- Grosor: 3px
- Tipo: Línea punteada
- Opacidad: 0.6

5. Trazos individuales por tramo:

- Líneas que conectan cada pedido con el siguiente
- Color: Según el estado del pedido **destino**
- Grosor: 5px
- Tipo: Línea sólida
- Opacidad: 0.8
- Ejemplo:
 - Pedido 1 (ACTIVO) → Pedido 2 (EN RUTA): Línea naranja
 - Pedido 2 (EN RUTA) → Pedido 3 (ENTREGADO): Línea verde

6. Botón "Center":

- Botón flotante en esquina superior derecha
- Al presionar, ajusta el zoom para mostrar todos los marcadores
- Útil si el usuario hizo zoom manualmente

6.2 Directions API de Mapbox

Para generar rutas reales por carreteras (no líneas rectas):

Proceso:

1. Construye URL de Directions API:

```
https://api.mapbox.com/directions/v5/mapbox/driving/{coordenadas}
?geometries=geojson&access_token={token}
```

2. Coordenadas en formato: `lng1,lat1;lng2,lat2;lng3,lat3`
3. **Límite:** Máximo 25 coordenadas por petición
4. Si hay más de 25 pedidos:

- Divide en segmentos de 25
- Genera rutas parciales
- Une los segmentos

Respuesta de Directions API:

- GeoJSON con geometría de la ruta
- Distancia total en metros
- Duración estimada en segundos
- Instrucciones turn-by-turn (opcional)

Renderizado en el mapa:

- Capa tipo **line** con el GeoJSON
- Sigue carreteras reales
- Considera sentido de las calles
- Evita rutas imposibles (ej: contra sentido)

6.3 Exportación a Google Maps

Genera una URL que abre directamente la ruta en Google Maps app.

Formato de URL:

```
https://www.google.com/maps/dir/?api=1
&travelmode=driving
&destination={lat_final},{lng_final}
&waypoints={lat1},{lng1}|{lat2},{lng2}|{lat3},{lng3}...
```

Parámetros:

- **api=1**: Versión de la API de URLs de Google Maps
- **travelmode=driving**: Modo de viaje en auto
- **destination**: Coordenadas del último pedido de la ruta
- **waypoints**: Coordenadas de todos los pedidos intermedios, separados por |

Límites de Google Maps:

- Máximo **20 waypoints** intermedios
- Si hay más de 20 pedidos:
 - Selecciona los primeros 20
 - Muestra advertencia al usuario
 - Sugiere dividir la ruta en segmentos

Proceso al exportar:

1. Usuario presiona botón "Abrir en Google Maps"
2. Sistema valida cantidad de pedidos
3. Si > 20: Muestra dialog de confirmación

4. Construye URL
5. Crea Intent con `ACTION_VIEW`
6. Android detecta apps compatibles:
 - Google Maps (preferida)
 - Waze
 - Navegador web
7. Usuario elige app
8. Google Maps abre con ruta trazada
9. Usuario puede iniciar navegación turn-by-turn

6.4 Parseo Flexible de Coordenadas

El sistema acepta múltiples formatos de coordenadas:

Formato 1: JSON objeto

```
{
  "lng": -99.1332,
  "lat": 19.4326
}
```

Formato 2: JSON string

```
"{\"lng\":-99.1332,\"lat\":19.4326}"
```

Formato 3: CSV (lat,lng)

```
19.4326, -99.1332
```

Formato 4: CSV (lng,lat)

```
-99.1332,19.4326
```

Clase de parseo:

```
class CoordenadasParser {
    public static LatLng parse(String coordStr) {
        // Intenta JSON primero
        if (coordStr.contains("{")) {
            JSONObject json = new JSONObject(coordStr);
            return new LatLng(json.getDouble("lat"), json.getDouble("lng"));
        }
    }
}
```

```

    // Intenta CSV
    String[] parts = coordStr.split(",");
    double val1 = Double.parseDouble(parts[0].trim());
    double val2 = Double.parseDouble(parts[1].trim());
    // Detecta cuál es lat y cuál lng (lat siempre entre -90 y 90)
    if (Math.abs(val1) <= 90) {
        return new LatLng(val1, val2); // lat, lng
    } else {
        return new LatLng(val2, val1); // lng, lat
    }
}
}

```

6.5 Generación Dinámica de HTML

La clase `HtmlBuilder` construye el HTML completo del mapa:

Estructura:

```

<!DOCTYPE html>
<html>
<head>
    <meta charset='utf-8' />
    <title>Mapa de Ruta</title>
    <meta name='viewport' content='width=device-width, initial-scale=1' />
    <script src='https://api.mapbox.com/mapbox-gl-js/v2.15.0/mapbox-gl.js'>
</script>
    <link href='https://api.mapbox.com/mapbox-gl-js/v2.15.0/mapbox-gl.css'
rel='stylesheet' />
    <style>
        body { margin: 0; padding: 0; }
        #map { position: absolute; top: 0; bottom: 0; width: 100%; }
        .marker { ... }
        .center-button { ... }
    </style>
</head>
<body>
    <div id='map'></div>
    <button class='center-button' onclick='centerMap()'>Center</button>
    <script>
        mapboxgl.accessToken = '{TOKEN}';
        const map = new mapboxgl.Map({
            container: 'map',
            style: 'mapbox://styles/mapbox/streets-v11',
            center: [{lng_promedio}, {lat_promedio}],
            zoom: 12
        });

        // Coordenadas de pedidos
        const pedidos = {PEDIDOS_JSON};
    </script>

```

```
// Agregar marcadores
pedidos.forEach((pedido, index) => {
  const el = document.createElement('div');
  el.className = 'marker';
  el.style.backgroundColor = getColorByEstado(pedido.estado);
  el.innerHTML = `${index + 1}`;

  new mapboxgl.Marker(el)
    .setLngLat([pedido.lng, pedido.lat])
    .setPopup(new mapboxgl.Popup().setHTML(`
      <strong>Factura:</strong> ${pedido.factura}<br>
      <strong>Cliente:</strong> ${pedido.cliente}<br>
      <strong>Dirección:</strong> ${pedido.direccion}
    `))
    .addTo(map);
});

// Dibujar ruta general (línea gris)
map.on('load', function() {
  map.addSource('ruta-general', {
    type: 'geojson',
    data: {
      type: 'Feature',
      geometry: {
        type: 'LineString',
        coordinates: {COORDENADAS_ARRAY}
      }
    }
  });
  map.addLayer({
    id: 'ruta-general',
    type: 'line',
    source: 'ruta-general',
    paint: {
      'line-color': '#9E9E9E',
      'line-width': 3,
      'line-dasharray': [2, 2],
      'line-opacity': 0.6
    }
  });

  // Dibujar tramos individuales
  {TRAMOS_INDIVIDUALES}

  // Ajustar bounds
  const bounds = new mapboxgl.LngLatBounds();
  pedidos.forEach(p => bounds.extend([p.lng, p.lat]));
  map.fitBounds(bounds, { padding: 50 });
});

function getColorByEstado(estado) {
  const colores = {
    'ACTIVO': '#1976D2',
    'EN RUTA': '#EF6C00',
  }
```

```

        'REPROGRAMADO': '#8E24AA',
        'EN TIENDA': '#FBC02D',
        'ENTREGADO': '#2E7D32',
        'CANCELADO': '#C62828'
    };
    return colores[estado] || '#757575';
}

function centerMap() {
    const bounds = new mapboxgl.LngLatBounds();
    pedidos.forEach(p => bounds.extend([p.lng, p.lat]));
    map.fitBounds(bounds, { padding: 50, duration: 1000 });
}
</script>
</body>
</html>

```

Ventajas del HTML dinámico:

- No requiere actividades adicionales
- Se renderiza en WebView
- Interacción fluida (zoom, pan, popups)
- Actualización en tiempo real
- Estilos personalizables

Token de Mapbox

Ubicación: `res/values/strings.xml`

```

<string name="mapbox_public_token">pk.eyJ1Ijo1VFVfVVNVQVJJTyIsImEiOiIjbjbGV...
</string>

```

Obtención del token:

1. Registrarse en <https://www.mapbox.com>
2. Ir a Account → Tokens
3. Crear nuevo token público
4. Copiar y pegar en `strings.xml`

Límites del plan gratuito:

- 50,000 cargas de mapa/mes
- 100,000 peticiones a Directions API/mes
- Suficiente para la mayoría de flotas

Layout

`dialog_grupo_mapa.xml`

Contenido:

```
<LinearLayout>
    <WebView
        android:id="@+id/webview_mapa"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />

    <Button
        android:id="@+id/btn_abrir_google_maps"
        android:text="Abrir en Google Maps"
        android:icon="@drawable/ic_google_maps" />

    <Button
        android:id="@+id/btn_cerrar"
        android:text="Cerrar" />
</LinearLayout>
```

 Experiencia de Usuario

1. Chofer abre **GrupoRutaActivity** con 8 pedidos
2. Presiona FAB "Ver Mapa"
3. Se abre diálogo a pantalla completa
4. WebView carga HTML con Mapbox
5. Aparecen 8 marcadores numerados:
 - [1] Azul (ACTIVO)
 - [2] Azul (ACTIVO)
 - [3] Naranja (EN RUTA)
 - [4] Azul (ACTIVO)
 - [5] Azul (ACTIVO)
 - [6] Verde (ENTREGADO)
 - [7] Azul (ACTIVO)
 - [8] Azul (ACTIVO)
6. Línea gris punteada conecta todos los pedidos
7. Tramos individuales tienen colores según destino
8. Chofer hace zoom en un marcador
9. Toca el marcador #3
10. Popup muestra:
 - Factura: #12345
 - Cliente: Farmacia 123
 - Dirección: Av. Reforma 456
11. Chofer explora el mapa
12. Presiona botón "Center" → Mapa se recentra
13. Presiona "Abrir en Google Maps"
14. Sistema genera URL con 8 waypoints
15. Abre Google Maps app

- 16. Ruta completa aparece trazada
- 17. Chofer presiona "Iniciar navegación"
- 18. Google Maps inicia guía turn-by-turn

7. Módulo de Vehículos

Ubicación

ui/vehicle/VehicleFragment.java

Funcionalidades

7.1 Consulta de Información del Vehículo

Muestra todos los detalles del vehículo asignado al chofer logueado.

Información visualizada:

Campo	Descripción
Tipo de vehículo	Ej: Camioneta, Van, Camión 3.5 ton
Placa	Número de matrícula
Número de serie (VIN)	Identificador único del vehículo
ID interno	ID en la base de datos
Sucursal	Sucursal a la que está asignado
Kilometraje actual	Último kilometraje registrado
Modelo	Marca y modelo (opcional)
Año	Año del vehículo (opcional)
Color	Color del vehículo (opcional)

Formato visual:

- Card grande con toda la información
- Ícono de vehículo en la parte superior
- Campos organizados en pares (label: valor)
- Diseño Material Design con elevación

7.2 Botón "Ver Inspección del Día"

- Ubicado en la parte inferior del card
- Al presionar:
 - Abre `InspeccionHoyActivity`
 - Muestra la inspección vehicular del día actual (solo lectura)
 - Si no hay inspección hoy, muestra mensaje

Utilidad:

- Consultar qué ítems se marcaron como "Mal"
- Ver observaciones registradas
- Evidencia en caso de incidentes

7.3 Mensaje si No Hay Vehículo Asignado

Si el usuario no tiene vehículo asignado:

- Muestra mensaje: "No tienes vehículo asignado. Contacta al administrador."
- Ícono de advertencia
- Botón "Cerrar sesión"

Nota: Esta situación también se maneja en MainActivity con un dialog bloqueante.

🔑 API Endpoint

```
GET /Pedidos_GA/App/Consultar.php?username={username}&v2=1
```

Response (JSON):

```
{
  "success": true,
  "vehiculo": {
    "id": int,
    "tipo": "string",
    "placa": "string",
    "numero_serie": "string",
    "sucursal": "string",
    "kilometraje_actual": int,
    "modelo": "string",
    "anio": int,
    "color": "string"
  }
}
```

📱 Layout

res/layout/vehiculos.xml

⚙️ Experiencia de Usuario

1. Chofer abre el drawer de navegación
2. Selecciona "Mi Vehículo"
3. Se carga **VehicleFragment**
4. Sistema consulta al servidor con el username
5. Servidor responde con los datos del vehículo
6. Se muestra card con:
 - Tipo: Camioneta

- Placa: ABC-123-XYZ
 - Serie: 1HGBH41JXMN109186
 - Sucursal: Centro
 - Kilometraje: 45,320 km
7. Chofer presiona "Ver Inspección del Día"
 8. Se abre `InspeccionHoyActivity`
 9. Ve checklist completo:
 - Sistema de Luces: Todos "Bien"
 - Parte Externa: Espejos "Mal" con observación "Espejo derecho roto"
 - etc.
 10. Cierra la inspección
 11. Regresa al fragment de vehículos
-

8. Sistema de Actualización Automática

Ubicación

`UpdateManager.java`

Funcionalidades

8.1 Verificación Automática al Iniciar

- Se ejecuta automáticamente al iniciar `MainActivity`
- Consulta al servidor: ¿Hay nueva versión disponible?
- Compara `versionCode` local vs. servidor
- Lógica:

```
int versionCodeLocal = BuildConfig.VERSION_CODE;
int versionCodeServidor = response.getInt("versionCode");

if (versionCodeServidor > versionCodeLocal) {
    // Hay actualización disponible
    mostrarDialogActualizacion();
}
```

8.2 Dialog Informativo

Cuando hay actualización disponible, muestra un dialog con:

Título: "Actualización Disponible"

Contenido:

- Versión actual: 1.2.3
- Nueva versión: 1.3.0
- Changelog:

- Modal obligatorio de kilometraje
- Formulario de condiciones del vehículo
- Función para generar servicio si el kilometraje es mayor
- Agrupación de facturas
- Gestión de mapa de ruta
- Exportar ruta a Google Maps
- Nuevo módulo de vehículo
- Arreglo de bugs con respecto a excepción de conexión

Botones:

- **"Actualizar ahora"** → Inicia descarga
- **"Más tarde"** → Cierra dialog, permite usar la app
- **"Cancelar"** → Cierra dialog

Configuración:

- Dialog **cancelable** (permite cerrar)
- No fuerza la actualización (opcional)
- Se puede configurar para **forzar** actualizaciones críticas

8.3 Descarga Automática

Proceso al presionar "Actualizar ahora":

1. Crea petición de descarga:

```
DownloadManager.Request request = new
DownloadManager.Request(Uri.parse(apkUrl));
request.setTitle("App Pedidos - Actualización");
request.setDescription("Descargando versión " + nuevaVersion);
request.setNotificationVisibility(DownloadManager.Request.VISIBILITY_VISIBLE
_NOTIFY_COMPLETED);
request.setDestinationInExternalPublicDir(Environment.DIRECTORY_DOWNLOADS,
"app_pedidos_" + nuevaVersion + ".apk");
```

2. Usa DownloadManager del sistema:

- Descarga en segundo plano
- Notificación de progreso en la barra de estado
- Maneja interrupciones (pérdida de conexión, batería baja)
- Reintentos automáticos

3. Configuración de descarga:

- Solo con WiFi (opcional, configurable)
- Permite descarga con datos móviles (actual)
- Requiere conexión estable

8.4 BroadcastReceiver para Instalación

Escucha el evento `ACTION_DOWNLOAD_COMPLETE`:

```
private BroadcastReceiver downloadCompleteReceiver = new BroadcastReceiver() {
    @Override
    public void onReceive(Context context, Intent intent) {
        long downloadId = intent.getLongExtra(DownloadManager.EXTRA_DOWNLOAD_ID,
-1);

        if (downloadId == downloadIdActual) {
            // Obtener URI del archivo descargado
            Uri apkUri = downloadManager.getUriForDownloadedFile(downloadId);

            // Crear Intent de instalación
            Intent installIntent = new Intent(Intent.ACTION_VIEW);
            installIntent.setDataAndType(apkUri, "application/vnd.android.package-
archive");
            installIntent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK |
Intent.FLAG_GRANT_READ_URI_PERMISSION);

            // Abrir instalador
            context.startActivity(installIntent);
        }
    }
};
```

Permisos necesarios:

- `REQUEST_INSTALL_PACKAGES` (Android 8.0+)
- `WRITE_EXTERNAL_STORAGE` (Android < 10)

8.5 Instalación del APK

1. DownloadManager termina la descarga
2. Notificación: "Descarga completada"
3. BroadcastReceiver detecta el evento
4. Abre intent de instalación
5. Android muestra pantalla de instalación:
 - Permisos solicitados
 - Botón "Instalar"
6. Usuario presiona "Instalar"
7. App se cierra automáticamente
8. Se instala la nueva versión
9. Ícono de la app permanece (actualización in-place)
10. Usuario abre la app nuevamente
11. Nueva versión está activa

8.6 Validación de Actualizaciones

Actualizaciones opcionales:

- Usuario puede posponer
- Dialog se muestra cada vez que abre la app
- No bloquea el uso

Actualizaciones forzadas (configurable):

- Dialog **no cancelable**
- Botón "Actualizar ahora" es la única opción
- Bloquea el uso hasta actualizar
- Útil para correcciones críticas de seguridad

Implementación de forzar actualización:

```
if (esCritica) {
    dialog.setCancelable(false);
    dialog.setButton(DialogInterface.BUTTON_NEGATIVE, "Actualizar ahora", ...);
    // No hay botón "Más tarde"
} else {
    dialog.setCancelable(true);
    dialog.setButton(DialogInterface.BUTTON_POSITIVE, "Actualizar ahora", ...);
    dialog.setButton(DialogInterface.BUTTON_NEUTRAL, "Más tarde", ...);
}
```

8.7 Cache y Frecuencia

- Verificación: Cada vez que se inicia MainActivity
- No hay cache local de versiones
- Consulta siempre al servidor
- Respuesta es ligera (< 1 KB)
- No afecta el rendimiento

🔗 API Endpoint

```
GET /Pedidos_GA/App/check_update.php
```

Response (JSON):

```
{
  "versionCode": 124,
  "versionName": "1.3.0",
  "esCritica": false,
  "changelog": "- Modal obligatorio de kilometraje\n- Formulario de condiciones del vehículo\n...",
  "apkUrl": "http://192.168.60.194/Pedidos_GA/App/apk/app_pedidos_v1.3.0.apk"
}
```

Campos:

- **versionCode** (int): Código numérico de versión (debe ser > que el local)
- **versionName** (string): Nombre legible de la versión (ej: 1.3.0)
- **esCritica** (boolean): Si es true, fuerza la actualización
- **changelog** (string): Descripción de los cambios (soporte de `\n` para saltos de línea)
- **apkUrl** (string): URL completa del archivo APK

🧑‍💻 Experiencia de Usuario

1. Chofer abre la app (versión 1.2.3)
2. MainActivity inicia
3. UpdateManager consulta servidor
4. Servidor responde: Nueva versión 1.3.0 disponible
5. Dialog aparece:

 Actualización Disponible

Versión actual: 1.2.3

Nueva versión: 1.3.0

Novedades:

- Modal obligatorio de kilometraje
- Formulario de condiciones del vehículo
- Función para generar servicio si el kilometraje es mayor
- Agrupación de facturas
- Gestión de mapa de ruta
- Exportar ruta a Google Maps
- Nuevo módulo de vehículo
- Arreglo de bugs

[Actualizar ahora] [Más tarde]

6. Chofer presiona "Actualizar ahora"
7. Notificación: "Descargando App Pedidos v1.3.0..."
8. Barra de progreso en la notificación
9. Descarga completa: Notificación cambia a "Descarga completada"
10. Instalador de Android se abre automáticamente
11. Pantalla: "¿Deseas instalar esta actualización?"
12. Chofer presiona "Instalar"
13. App se cierra
14. Instalación en progreso (5-10 segundos)
15. Instalación completa
16. Chofer abre la app nuevamente
17. Splash screen muestra: "Versión 1.3.0"
18. App funciona con las nuevas funcionalidades

9. Restricciones de la Aplicación

Ubicación

MainActivity.java (lógica de bloqueo y validaciones)

Funcionalidades

9.1 Restricción: Vehículo No Asignado

Validación:

```
private void verificarVehiculoAsignado() {
    String url = ApiConfig.getBaseUrl() + "/Pedidos_GA/App/verificar_vehiculo.php?
    username=" + username;

    // Petición al servidor
    // Si vehiculo_id == null o vehiculo_id == 0:
    mostrarDialogoSinVehiculo();
}
```

Dialog mostrado:

- **Título:** "Sin Vehículo Asignado"
- **Mensaje:** "No tienes un vehículo asignado. No puedes usar la aplicación hasta que el administrador te asigne uno."
- **Ícono:**  Advertencia
- **Botón único:** "Cerrar Sesión"
- **Cancelable:** NO (no se puede cerrar tocando afuera o presionando Atrás)

Comportamiento:

- Bloquea completamente el acceso a la app
- DrawerLayout permanece bloqueado (LOCK_MODE_LOCKED_CLOSED)
- No permite navegar a ningún fragment
- Única acción posible: Cerrar sesión

Lógica de cierre de sesión:

```
private void cerrarSesion() {
    SharedPreferences prefs = getSharedPreferences("MisPreferencias",
    MODE_PRIVATE);
    prefs.edit().clear().apply();

    Intent intent = new Intent(this, LoginActivity.class);
    intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK |
    Intent.FLAG_ACTIVITY_CLEAR_TASK);
    startActivity(intent);
}
```



```
    finish();  
}
```

9.2 Restricción: Cambio de Vehículo

Detección:

```
private void detectaCambioVehiculo() {  
    String vehiculoIdAlmacenado = prefs.getString("vehiculo_id", null);  
    String vehiculoIdServidor = response.getString("vehiculo_id");  
  
    if (!vehiculoIdAlmacenado.equals(vehiculoIdServidor)) {  
        mostrarDialogoCambioVehiculoYSalir();  
    }  
}
```

Dialog mostrado:

- **Título:** "Cambio de Vehículo Detectado"
- **Mensaje:** "Tu vehículo asignado ha cambiado. Por seguridad, debes iniciar sesión nuevamente."
- **Ícono:**  Cambio
- **Botón único:** "Aceptar"
- **Cancelable:** NO

Comportamiento:

1. Detecta que **vehiculo_id** en BD es diferente al almacenado en SharedPreferences
2. Muestra dialog informativo
3. Al presionar "Aceptar":
 - Limpia SharedPreferences
 - Cierra sesión
 - Redirige a LoginActivity
4. Usuario debe volver a loguearse
5. Al loguearse, se actualiza el **vehiculo_id** en SharedPreferences
6. Nueva inspección vehicular será del nuevo vehículo

Razón de esta restricción:

- Evita confusión en inspecciones (mezclar datos de dos vehículos)
- Asegura que el kilometraje registrado corresponde al vehículo correcto
- Auditoría clara de qué chofer usó qué vehículo

9.3 Restricción: Kilometraje No Registrado

Validación:

```
private void verificarEstadoKilometraje() {
    // Obtener hora actual
    Calendar ahora = Calendar.getInstance();
    int hora = ahora.get(Calendar.HOUR_OF_DAY);
    int diaSemana = ahora.get(Calendar.DAY_OF_WEEK);

    // Solo después de las 8 AM y no en domingo
    if (hora >= 8 && diaSemana != Calendar.SUNDAY) {
        // Consultar servidor: ¿Ya registró km hoy?
        // Si debe_capturar == true:
        mostrarDialogoCapturaKilometraje();
    }
}
```

Dialog mostrado:

- **Título:** "Registro de Kilometraje"
- **Mensaje:** "Por favor registra el kilometraje inicial del día antes de continuar."
- **Campo:** Input numérico para ingresar kilometraje
- **Fecha y hora:** Auto-llenada, no editable
- **Botón:** "Registrar"
- **Cancelable:** NO

Validación de entrada:

```
int kmIngresado = Integer.parseInt(editTextKm.getText().toString());
int lastKmFinal = prefs.getInt("lastKmFinal", 0);

if (kmIngresado < lastKmFinal) {
    Toast.makeText(this, "El kilometraje no puede ser menor al último registrado (" + lastKmFinal + " km)", Toast.LENGTH_LONG).show();
    return; // No permite continuar
}
```

Comportamiento:

- Bloquea completamente la UI hasta ingresar kilometraje
- DrawerLayout permanece bloqueado
- No permite cerrar el dialog
- Al registrar exitosamente:
 - Guarda `lastKmFinal` en SharedPreferences
 - Cierra el dialog
 - Continúa con la siguiente validación (inspección)

Manejo de errores:

- Si falla el registro por error de red:
 - Muestra dialog: "Error al registrar"

- Botones: "Reintentar" | "Cerrar Sesión"
- "Reintentar" → Vuelve a intentar el registro
- "Cerrar Sesión" → Cierra sesión y regresa a login

9.4 Restricción: Inspección Diaria Pendiente

Validación:

```
private void verificarInspeccionHoyYQuizasLanzar() {
    Calendar ahora = Calendar.getInstance();
    int hora = ahora.get(Calendar.HOUR_OF_DAY);
    int diaSemana = ahora.get(Calendar.DAY_OF_WEEK);

    if (hora >= 8 && diaSemana != Calendar.SUNDAY) {
        // Consultar servidor: ¿Ya hizo inspección hoy?
        String url = ApiConfig.getBaseUrl() +
        "/Pedidos_GA/App/obtener_checklist_hoy.php?username=" + username + "&fecha=" +
        fechaHoy;

        // Si no existe inspección:
        lanzarInspeccionVehicular();
    }
}
```

Proceso:

1. Consulta al servidor: ¿Existe inspección para hoy?
2. Si NO existe:
 - Lanza `InspeccionVehicularActivity` con `startActivityForResult()`
 - `MainActivity` queda en pausa
 - Usuario completa el formulario de inspección
 - Al completar, `InspeccionVehicularActivity` cierra con `RESULT_OK`
 - `MainActivity` recibe el resultado en `onActivityResult()`
 - Desbloquea el `DrawerLayout`
3. Si existe:
 - No lanza nada
 - Desbloquea el `DrawerLayout` directamente

Manejo de errores:

- Si el usuario cierra la Activity de inspección sin completarla:
 - `onActivityResult()` recibe `RESULT_CANCELED`
 - Muestra dialog: "Debes completar la inspección para usar la app"
 - Botones: "Reintentar" | "Cerrar Sesión"
- Si falla la consulta al servidor:
 - Muestra dialog de error
 - Botones: "Reintentar" | "Cerrar Sesión"

9.5 Bloqueo del DrawerLayout

Estados del DrawerLayout:

```
// Estado BLOQUEADO (por defecto)
drawerLayout.setDrawerLockMode(DrawerLayout.LOCK_MODE_LOCKED_CLOSED);
// - No se puede abrir deslizando desde el borde
// - No se puede abrir con el botón hamburguesa
// - No responde a gestos táctiles

// Estado DESBLOQUEADO (tras pasar todas las validaciones)
drawerLayout.setDrawerLockMode(DrawerLayout.LOCK_MODE_UNLOCKED);
// - Se puede abrir deslizando desde el borde izquierdo
// - Se puede abrir con el botón hamburguesa
// - Permite navegación normal
```

Método de desbloqueo:

```
private void desbloquearUsoApp() {
    DrawerLayout drawerLayout = findViewById(R.id.drawer_layout);
    drawerLayout.setDrawerLockMode(DrawerLayout.LOCK_MODE_UNLOCKED);

    Toast.makeText(this, "Aplicación desbloqueada. Puedes comenzar a trabajar.",
        Toast.LENGTH_SHORT).show();
}
```

Llamado solo cuando:

- ☒ Vehículo está asignado
- ☒ Kilometraje del día está registrado
- ☒ Inspección vehicular del día está completa

9.6 Flujo Completo de Validaciones

Orden de ejecución en `onCreate()` de `MainActivity`:

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    // 1. Configurar UI
    setupDrawerLayout();
    setupNavigation();

    // 2. Verificar vehículo asignado (PRIMERA VALIDACIÓN)
    verificarVehiculoAsignado(); // Si falla, bloquea y muestra dialog → NO
    continúa
}
```

```

// 3. Verificar actualización disponible (no bloquea)
UpdateManager.verificarActualizacion(this);

// 4. Verificar kilometraje (SEGUNDA VALIDACIÓN)
verificarEstadoKilometraje(); // Si debe capturar, muestra dialog bloqueante

// 5. Verificar inspección (TERCERA VALIDACIÓN)
verificarInspeccionHoyYQuizasLanzar(); // Si no existe, lanza Activity

// 6. Si todo OK, desbloquear
desbloquearUsoApp();
}

```

Diagrama de flujo:

```

Inicio de MainActivity
    ↓
¿Tiene vehículo asignado?
    NO → Dialog bloqueante → Solo opción: Cerrar sesión → Fin
    SÍ ↓
        ↓
¿Cambió el vehículo?
    SÍ → Dialog → Cerrar sesión → Fin
    NO ↓
        ↓
¿Es después de las 8 AM y no es domingo?
    NO → Desbloquear app → Uso normal
    SÍ ↓
        ↓
¿Ya registró kilometraje hoy?
    NO → Dialog bloqueante de km → Esperar registro
    SÍ ↓
        ↓
¿Ya hizo inspección hoy?
    NO → Lanzar InspeccionVehicularActivity → Esperar completar
    SÍ ↓
        ↓
Desbloquear DrawerLayout
    ↓
Uso normal de la app

```

API Endpoints

Verificar Vehículo

GET /Pedidos_GA/App/verificar_vehiculo.php?username={username}

Response (JSON):

```
{
  "tiene_vehiculo": boolean,
  "vehiculo_id": int,
  "cambio_vehiculo": boolean
}
```

Estado de Kilometraje

GET /Pedidos_GA/App/estado_kilometraje.php?username={username}

Response (JSON):

```
{
  "debe_capturar": boolean,
  "ultimo_km": int,
  "fecha_ultimo_registro": "YYYY-MM-DD"
}
```

Estado de Inspección

GET /Pedidos_GA/App/obtener_checklist_hoy.php?username={username}&fecha={YYYY-MM-DD}

Response (JSON):

```
{
  "success": boolean,
  "existe": boolean,
  "items": [...]
}
```

🧑‍💻 Experiencia de Usuario - Caso Exitoso

1. Chofer abre la app a las 8:30 AM (Lunes)
2. MainActivity inicia
3. ☒ Verificación 1: Tiene vehículo asignado (ID 5)
4. ☒ Verificación 2: Vehículo no cambió (sigue siendo ID 5)
5. 🔍 Verificación 3: Es 8:30 AM y es lunes → Debe capturar km
6. 🔍 Consulta servidor: ¿Ya registró km hoy? → NO
7. 📝 Muestra dialog de kilometraje
8. Chofer ingresa: 45,320 km
9. Sistema valida: 45,320 >= 45,100 (último km) ☒
10. Envía al servidor → Success
11. Guarda lastKmFinal = 45,320
12. 🔍 Verificación 4: ¿Ya hizo inspección hoy? → NO
13. 📄 Lanza InspeccionVehicularActivity
14. Chofer completa checklist de 29 ítems

15. Presiona "Enviar"
16. Envía al servidor → Success
17. InspeccionVehicularActivity cierra con RESULT_OK
18. MainActivity recibe resultado exitoso
19.  Desbloquea DrawerLayout
20. Toast: "Aplicación desbloqueada. Puedes comenzar a trabajar."
21. Chofer puede abrir el drawer
22. HomeFragment carga pedidos activos
23. Chofer comienza su jornada

Experiencia de Usuario - Caso con Error

1. Chofer abre la app a las 8:30 AM
2. MainActivity inicia
3. ☒ Verificación 1: Tiene vehículo asignado
4. ☒ Verificación 2: Vehículo no cambió
5.  Verificación 3: Debe capturar km
6.  Muestra dialog de kilometraje
7. Chofer ingresa: 45,320 km
8. Sistema intenta enviar al servidor
9.  Error de red: "Timeout"
10. Dialog de error: "No se pudo registrar el kilometraje. Verifica tu conexión."
11. Botones: [Reintentar] [Cerrar Sesión]
12. Chofer presiona "Reintentar"
13. Vuelve a mostrar dialog de kilometraje
14. Chofer ingresa 45,320 nuevamente
15. Envío exitoso
16. Continúa con inspección...

Módulos Secundarios

A. HomeFragment - Pedidos Activos

Ubicación

ui/home/HomeFragment.java

Funcionalidades

- **Lista de pedidos activos:**
 - RecyclerView con cards
 - Muestra solo pedidos del chofer logueado
 - Estados: ACTIVO, EN RUTA, REPROGRAMADO
- **Actualización automática:**
 - Polling cada 5 segundos (configurable)

- Pull-to-refresh manual
- BroadcastReceiver escucha cambios
- **Notificaciones persistentes:**
 - Crea notificación para cada pedido "EN RUTA"
 - Notificación contiene:
 - Cliente
 - Dirección
 - Botón "Abrir Detalle"
 - Botón "Ver en Mapa"
- **Click en card:**
 - Abre `DetallePedidoActivity`

Layout

`fragment_home.xml`

B. SlideshowFragment - Historial

Ubicación

`ui/slideshow/SlideshowFragment.java`

Funcionalidades

- **Lista de pedidos completados:**
 - Estados: ENTREGADO, CANCELADO
 - Orden: Más reciente primero
- **Filtro por fecha:**
 - DatePicker para seleccionar rango
 - Por defecto: Últimos 7 días
- **Información resumida:**
 - Cliente
 - Fecha de entrega
 - Estado final
- **Click en card:**
 - Abre `DetallePedidoActivity` (solo lectura)

Layout

`fragment_slideshow.xml`

C. EstadísticasFragment - Métricas

Ubicación

ui/Estadísticas/EstadísticasFragment.java

Funcionalidades

- **Gráficos con MPAndroidChart:**
 - Pedidos entregados por día (BarChart)
 - Pedidos por estado (PieChart)
 - Tendencia semanal (LineChart)
- **Métricas numéricas:**
 - Total de entregas del mes
 - Tasa de éxito (%)
 - Entregas a tiempo vs. retrasadas
- **Filtros:**
 - Por rango de fechas
 - Por sucursal (si aplica)

Layout

fragment_estadisticas.xml

Configuración del Servidor

URL Base del Servidor

Archivo: ApiConfig.java

```
public class ApiConfig {  
    // URL base del servidor  
    private static final String BASE_URL = "http://192.168.60.194";  
  
    // Path base de la API  
    private static final String API_PATH = "/Pedidos_GA/App/";  
  
    public static String getBaseUrl() {  
        return BASE_URL + API_PATH;  
    }  
}
```

Para cambiar el servidor:

1. Abrir `ApiConfig.java`
 2. Modificar `BASE_URL` con la nueva IP o dominio
 3. Recompilar la app
-

Encoding UTF-8 Personalizado

Para asegurar que los caracteres especiales (ñ, acentos, etc.) se manejen correctamente:

Ubicación: `network/`

- `Utf8JsonObjectRequest.java`
- `Utf8JsonArrayRequest.java`
- `Utf8StringRequest.java`

Uso:

```
Utf8JsonObjectRequest request = new Utf8JsonObjectRequest(  
    Request.Method.POST,  
    url,  
    jsonBody,  
    response -> { /* Success */ },  
    error -> { /* Error */ }  
);
```

Configuración de Cleartext Traffic

Archivo: `AndroidManifest.xml`

```
<application  
    android:usesCleartextTraffic="true"  
    ...>
```

 **Importante:**

- Solo para desarrollo/debug
 - En producción, usar HTTPS
-

Permisos Requeridos

Archivo: `AndroidManifest.xml`

```
<!-- Red -->  
<uses-permission android:name="android.permission.INTERNET" />  
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

```
<!-- Ubicación -->
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

<!-- Notificaciones (Android 13+) -->
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

<!-- Almacenamiento para fotos y descargas -->
<uses-permission android:name="android.permission.READ_MEDIA_IMAGES" />
<uses-permission android:name="android.permission.READ_MEDIA_VIDEO" />
<uses-permission android:name="android.permission.READ_MEDIA_AUDIO" />
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"
    android:maxSdkVersion="32" />
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"
    android:maxSdkVersion="29" />

<!-- Instalación de APKs (para actualizaciones) -->
<uses-permission android:name="android.permission.REQUEST_INSTALL_PACKAGES" />
```

Permisos solicitados en tiempo de ejecución:

- Ubicación (al actualizar estado de pedido)
- Notificaciones (al crear pedidos "EN RUTA")
- Almacenamiento (al subir foto de evidencia)

API Endpoints

Resumen de Todos los Endpoints

Endpoint	Método	Descripción
/login.php	POST	Autenticación de usuario
/verificar_vehiculo.php	GET	Verificar si tiene vehículo asignado
/estado_kilometraje.php	GET	Consultar si debe capturar km
/registrar_kilometraje.php	POST	Registrar kilometraje del día
/obtener_checklist_hoy.php	GET	Obtener inspección del día
/guardar_checklist.php	POST	Guardar inspección vehicular
/Consultar.php	GET	Obtener pedidos (activos/grupo/vehículo)
/actualizar_estado.php	POST	Cambiar estado de pedido
/subir_foto.php	POST	Subir foto de evidencia
/verificar_foto.php	GET	Verificar si existe foto
/descargar_documento.php	GET	Descargar PDF de factura
/historial_pedido.php	GET	Obtener timeline de cambios

Endpoint	Método	Descripción
/check_update.php	GET	Verificar si hay actualización

Base URL: http://192.168.60.194/Pedidos_GA/App/

Flujo de Trabajo del Usuario

Inicio del Día

-  **8:00 AM** - Chofer abre la app
-  Si no hay sesión → Login
-  Sistema verifica vehículo asignado
-  **Modal de kilometraje** aparece:
 - Ingresa kilometraje inicial (ej: 45,320 km)
 - Sistema valida que sea $\geq$ último registrado
 - Envía al servidor
-  **Formulario de inspección** se abre automáticamente:
 - Revisa 29 ítems del vehículo
 - Marca secciones completas con botones radio
 - Agrega observaciones si algo está "Mal"
 - Envía checklist al servidor
-  **App se desbloquea** → Drawer disponible
-  HomeFragment carga pedidos del día

Gestión de Pedidos

-  Chofer ve lista de pedidos activos:
 - 5 pedidos en "Ruta Norte - Lunes AM"
-  Toca el grupo → [GrupoRutaActivity](#)
-  Presiona "Ver Mapa del Grupo"
-  Mapa Mapbox muestra todos los pedidos numerados
-  Presiona "Abrir en Google Maps"
-  Google Maps abre con ruta completa
-  Inicia navegación turn-by-turn

En Ruta

-  Chofer sigue la ruta a primer pedido
-  Notificación persistente: "Pedido #101 - EN RUTA"
-  Al llegar, regresa a la app
-  Abre detalle del pedido #101
-  Cambia estado: ACTIVO → EN RUTA
-  Toma foto de comprobante de entrega
-  Sube foto al servidor
-  Cambia estado: EN RUTA → ENTREGADO

23.  Card del pedido se pone verde
24.  Regresa al grupo → Siguiente pedido

Pedido Problemático

25.  Llega a pedido #103
26.  Cliente no está disponible
27.  Llama al cliente → Reagendar
28.  Cambia estado: EN RUTA → REPROGRAMADO
29.  Agrega observación: "Cliente pidió reprogramar para mañana"
30.  Card del pedido se pone morado
31.  Continúa con siguiente pedido

Fin del Día

32. ☒ Completa 4 de 5 pedidos (1 reprogramado)
33.  Abre "Estadísticas" en el drawer
34.  Ve gráfico: 4 entregas exitosas hoy
35.  Tasa de éxito: 80%
36.  Revisa historial de entregas de la semana
37.  Cierra la app

Día Siguiente

38.  **8:00 AM** - Abre la app
39. ☒ Sesión activa → No solicita login
40.  **Modal de kilometraje** aparece automáticamente
41.  Ingresa nuevo kilometraje (ej: 45,550 km)
42. ☒ Sistema valida: 45,550 >= 45,320 ☒
43.  **Formulario de inspección** se abre
44. ☒ Completa checklist
45.  App se desbloquea
46.  Ve pedidos del nuevo día + pedido #103 reprogramado
47.  Continúa el ciclo...

Notas Adicionales

Seguridad

- **✗ No hay autenticación con token JWT** (mejora pendiente)
-  **Cleartext HTTP** (cambiar a HTTPS en producción)
- ☒ Validación de permisos en tiempo de ejecución
- ☒ SharedPreferences sin encriptación (datos no sensibles)

Mejoras Sugeridas

- 1. Implementar autenticación con JWT
- 2. Migrar a HTTPS
- 3. Agregar cache offline con Room Database
- 4. Sincronización en segundo plano con WorkManager
- 5. Push notifications con FCM
- 6. Firma digital del chofer para entregas
- 7. QR de confirmación de entrega
- 8. Chat en vivo con despacho

 Métricas de Uso

- **Tamaño de APK:** ~15 MB
- **Permisos solicitados:** 11
- **Pantallas principales:** 8
- **API Endpoints:** 13
- **Dependencias externas:** 12

Contacto y Soporte

Desarrollador: [Alex Casillas y Julio Rubio] **Email:** [a.casillas@grupoascencio.com.mx o j.rubio@grupoascencio.com.mx] **Servidor:** pedidos.grupoascencio.com.mx

Última actualización: 2025-12-02 **Versión del documento:** 1.0 **Versión de la app:** 1.3.0
